**PART A – THEORY**

**1. As-Is vs. To-Be Process Models**

The distinction between **As-Is** and **To-Be** models is the foundation of Business Process Reengineering (BPR).

- **As-Is Process Model:** This represents the current state of the organization. It documents how a process actually functions today, including all manual steps, redundancies, and inefficiencies. Its primary goal is to provide a baseline for identifying "pain points."
- **To-Be Process Model:** This represents the future state. it documents the optimized process after improvements, such as automation or the introduction of a new system, have been implemented.

**Why they should never be mixed:** Mixing these models in a single diagram creates **structural ambiguity**. If a diagram contains both current manual steps and future automated steps without distinction:

- **Stakeholders** will be confused about what they are currently responsible for versus what is changing.
- **Developers** may accidentally build "legacy" inefficiencies into a new system.
- **Gap Analysis** becomes impossible, as you cannot clearly see the distance between the current problem and the proposed solution.

**2. UML Use Case vs. Activity Diagrams**

Moving from process understanding to system design requires both diagrams because they provide two different "views" of the system—**Scope** and **Logic**.

**Use Case Diagrams (The "What")**

Use Case diagrams define the **functional requirements** and the system boundary. They identify the "Actors" (users or external systems) and the specific goals (Use Cases) those actors need to achieve.

- **Role:** It answers *what* the system does and *who* interacts with it.
- **Why it's required:** It prevents "scope creep" by defining exactly what is (and isn't) part of the system.

**Activity Diagrams (The "How")**

Activity diagrams model the **dynamic behavior** and procedural flow of a process. They show the sequential flow of activities, decision points (branching), and parallel processing.

- **Role:** It answers *how* a specific Use Case is executed step-by-step.
- **Why it's required:** A Use Case only names a goal (e.g., "Withdraw Cash"); the Activity Diagram maps the logic required to achieve it (e.g., Check Balance -> If Sufficient -> Dispense Cash).